

Python Web Vulnerability Scanner

Code Documentation

A modular defensive web vulnerability scanner for authorized testing environments. The project analyzes HTTP security headers, cookies, information disclosure, and basic HTML form security.

Project Structure

```
web-vulnerability-scanner/  
├──  
│   ├── scanner.py  
│   ├── requirements.txt  
│   └── README.md  
├──  
│   └── modules/  
│       ├── __init__.py  
│       ├── headers.py  
│       ├── cookies.py  
│       ├── information.py  
│       └── forms.py  
├──  
└── reports/  
    └── report.json
```

requirements.txt

```
requests  
beautifulsoup4
```

scanner.py

```
import argparse
import json
import requests

from modules.headers import check_headers
from modules.cookies import check_cookies
from modules.information import check_information
from modules.forms import check_forms

def scan_website(url):
    print("=" * 60)
    print("PYTHON WEB VULNERABILITY SCANNER")
    print("=" * 60)

    print(f"\nTarget: {url}")
    print("Scanning...\n")

    findings = []

    try:
        response = requests.get(
            url,
            timeout=10,
            allow_redirects=True,
            headers={
                "User-Agent": "StudentSecurityScanner/1.0"
            }
        )

        print(f"Status Code : {response.status_code}")
        print(f"Final URL : {response.url}")

    except requests.RequestException as error:
        print(f"[ERROR] Could not connect to target: {error}")
        return

    findings.extend(check_headers(response))
    findings.extend(check_cookies(response))
    findings.extend(check_information(response))
    findings.extend(check_forms(response))

    print("\n" + "=" * 60)
    print("SCAN RESULTS")
    print("=" * 60)

    if not findings:
        print("\nNo findings detected.")
    else:
        for index, finding in enumerate(findings, start=1):
            print(
                f"\n[{index}] {finding['severity']} - "
                f"{finding['title']}"
            )
            print(f"    {finding['description']}")
            print(f"    Recommendation: {finding['recommendation']}")

    report = {
        "target": url,
        "final_url": response.url,
        "status_code": response.status_code,
        "findings": findings
    }

    with open("reports/report.json", "w") as file:
        json.dump(report, file, indent=4)

    print("\nReport saved to reports/report.json")

def main():
    parser = argparse.ArgumentParser(
        description="Authorized web vulnerability scanner"
    )

    parser.add_argument(
        "url",
        help="URL of the authorized web application"
    )

    args = parser.parse_args()
```

```
scan_website(args.url)
```

```
if __name__ == "__main__":  
    main()
```

modules/headers.py

```
def check_headers(response):

    findings = []

    security_headers = {
        "Content-Security-Policy":
            "Missing Content Security Policy header.",

        "X-Content-Type-Options":
            "Missing X-Content-Type-Options header.",

        "X-Frame-Options":
            "Missing X-Frame-Options header.",

        "Referrer-Policy":
            "Missing Referrer-Policy header.",

        "Permissions-Policy":
            "Missing Permissions-Policy header."
    }

    for header, description in security_headers.items():

        if header not in response.headers:

            findings.append({
                "category": "Security Headers",
                "severity": "MEDIUM",
                "title": f"Missing {header}",
                "description": description,
                "recommendation":
                    f"Configure the {header} HTTP response header."
            })

    return findings
```

modules/cookies.py

```
def check_cookies(response):

    findings = []

    cookies = response.cookies

    for cookie in cookies:

        if not cookie.secure:
            findings.append({
                "category": "Cookies",
                "severity": "MEDIUM",
                "title": "Cookie missing Secure flag",
                "description":
                    f"Cookie '{cookie.name}' does not have "
                    "the Secure attribute.",
                "recommendation":
                    "Set the Secure attribute for sensitive cookies."
            })

        if not cookie.has_nonstandard_attr("HttpOnly"):
            findings.append({
                "category": "Cookies",
                "severity": "MEDIUM",
                "title": "Cookie missing HttpOnly flag",
                "description":
                    f"Cookie '{cookie.name}' does not have "
                    "the HttpOnly attribute.",
                "recommendation":
                    "Set HttpOnly for cookies that do not need "
                    "client-side JavaScript access."
            })

    return findings
```

modules/information.py

```
def check_information(response):

    findings = []

    server_header = response.headers.get("Server")

    if server_header:
        findings.append({
            "category": "Information Disclosure",
            "severity": "LOW",
            "title": "Server information exposed",
            "description":
                f"The server header reveals: {server_header}",
            "recommendation":
                "Consider minimizing unnecessary server information."
        })

    powered_by = response.headers.get("X-Powered-By")

    if powered_by:
        findings.append({
            "category": "Information Disclosure",
            "severity": "LOW",
            "title": "Technology information exposed",
            "description":
                f"X-Powered-By reveals: {powered_by}",
            "recommendation":
                "Remove unnecessary technology-identifying headers."
        })

    return findings
```

modules/forms.py

```
from bs4 import BeautifulSoup

def check_forms(response):

    findings = []

    soup = BeautifulSoup(response.text, "html.parser")
    forms = soup.find_all("form")

    for form in forms:

        method = form.get("method", "GET").upper()

        if method == "GET":

            password_fields = form.find_all(
                "input",
                {"type": "password"}
            )

            if password_fields:
                findings.append({
                    "category": "Form Security",
                    "severity": "MEDIUM",
                    "title":
                        "Password form uses GET method",
                    "description":
                        "A form containing a password field "
                        "uses GET. Form data may appear in URLs.",
                    "recommendation":
                        "Use POST for forms containing sensitive data."
                })

    return findings
```

Example command

```
# Install dependencies
pip install -r requirements.txt

# Scan a local/authorized application
python scanner.py http://127.0.0.1:5000

# Example authorized test target
python scanner.py https://your-authorized-test-site.example
```

Example report.json

```
{
  "target": "http://127.0.0.1:5000",
  "final_url": "http://127.0.0.1:5000",
  "status_code": 200,
  "findings": [
    {
      "category": "Security Headers",
      "severity": "MEDIUM",
      "title": "Missing Content-Security-Policy",
      "description": "Missing Content Security Policy header.",
      "recommendation":
        "Configure the Content-Security-Policy HTTP response header."
    }
  ]
}
```

Important Usage Note

Use this scanner only against applications you own or have explicit permission to test. It is designed for defensive security assessment and does not attempt to exploit vulnerabilities.